Welcome to your next lesson: If/else: syntax, indentation, mental model.

I'm getting your personalized lesson ready—based on what you've learned, what's ahead, and how you learn best.

Step by step, you're moving forward.

- ## Imagine you're selling movie tickets ?

Rule in plain English:

- "If age is less than 12 → child price
- Otherwise → regular price"

That "if … otherwise …" is exactly what we'll teach Python to do.

Quick check for you:

If a kid is 10 years old, which price should they get?

You can just answer in words: "child price" or "regular price".

child price

Exactly — child price ✅

That "check something, then choose what to do" is what `if` does in Python.

Here's the same idea in code form, but I'll keep it small:

```python
age = 10
if age < 12:
    print("child price")
```

Key new thing: the indent (spaces) before `print(...)` shows Python "this line belongs to the `if`."

Does this basic idea make sense so far:

"Python checks `age < 12`, and only if it's true, it runs the indented line"?

Yes, that makes sense

Nice ?

Take a look at the little flow diagram below ? — it shows:

- Python checks `age < 12`
- If the answer is "yes", it goes into the indented block and runs `print("child price")`
- If the answer is "no", it skips that block

Next step: I want to show you how we add an `else` for the "otherwise: regular price" part.

Ready to see that?

Python if: one-way decision flow



Yes, I'm ready

Cool, let’s add the otherwise part now ?

```python
age = 10

if age < 12:
    print("child price")
else:
    print("regular price")
```

Step by step:

1. Python checks `age < 12`.
2. If that is true → it runs the first indented line.
3. If that is false → it skips that and runs the indented line under `else`.

Notice both `print` lines are indented the same amount (they are each inside their block).

Quick mental test (no code editor yet):

If `age = 15`, which line will run:

`print("child price")` or `print("regular price")`?

print("regular price")

Exactly right ?

Because `age = 15`:

- The check `age < 12` is `15 < 12` → that's false
- So Python skips the `if` block and runs the `else` block → `print("regular price")`

So in every `if`/`else`, only one of the two blocks runs.

Next, I want to zoom in on the indentation rule (those spaces before `print`), because Python is very strict about it.

Quick question first: have you ever seen an IndentationError in Python before, or is this your first time really thinking about indentation?

yes ive run into it and that pytthon is strict about sspecial 4 space indents easiest to use python specific code editors like VS coe with python etentions that default 40 4 space indents VS default

Nice, that's a solid instinct ?

In this course, the rule I want you to lock in is:

A code block in Python = a group of lines that all start with the same indentation.

For our `if`/`else`:

```python
if age < 12:
    print("child price")
else:
    print("regular price")
```

Visually:

- `if age < 12:` and `else` start at the left edge (no indent) → they're not inside anything.
- Both `print(...)` lines have the same 4-space indent → each one belongs to its own block.

If one `print` had 4 spaces and the other had 3 or 5, Python would complain.

Make sense so far: "everything in the block must line up under each other with the same indent"?

Yes, that makes sense

What do you think? ?

Look at this code:

```
age = 20

if age < 18:
    print("minor")
else:
    print("adult")
```

Which line is inside the `else` block (part of the `else`)?

A.

```
age = 20
```

B.

```
if age < 18:
```

C.

```
print("minor")
```

D.

```
print("adult")
```

Nice work on that one ?

You picked `print("adult")`, and that's exactly right because:

- The `else` line ends with a colon `:`.
- The line directly under it that is indented (`print("adult")`) is part of the `else` block.
- The other lines (`age = 20`, `if age < 18:`, `print("minor")`) are either before the `else` or belong to the `if` block.

Now I want to zoom in on the symbols we use in the `if` condition, like `<`, `>`, `==`.

Are you comfortable so far with `<` and `>` from regular math, or do you want a 20-second refresh on them in Python terms?

I'm comfortable with them

Perfect, that helps a lot ?

Let's lock in one more key symbol pair: `=` vs `==`.

- `=` means "put this value into the variable" (assignment)

```
age = 20      # set age to 20
```

- `==` means "is this equal to…?" (a question used in `if`)

```
if age == 20:
    print("age is twenty")
```

So: `=` changes a value, `==` just checks a value.

Quick check in your own words (short is fine):

What's the role of `=` compared to `==`?

= assigns a value == compares a value

Exactly right ?

- `=` → assigns a value to a variable (`age = 20`)
- `==` → compares values in a condition (`if age == 20:`)

That distinction is super important for `if` statements.

Next, I want you to try a tiny `if`/`else` yourself in the code editor:

Make a program that:

- has a variable `score`

- if score >= 60 it prints "pass"
- otherwise it prints "fail"

Write this in the code editor, run it, and when it works, hit Submit code so I can check it.

Python

```
score = 70
if score >= 60:
    print("pass")
else:
    print("fail")
```

Nice job, Ron — that's exactly what I wanted ✅

Your code:

Python

```
score = 70
if score >= 60:
    print("pass")
else:
    print("fail")
```

Step by step:

- score = 70 → you assign 70 to score (good use of =).
- if score >= 60: → you compare using >= (good use of a comparison operator).
- Indentation is correct: 4 spaces under if, 4 spaces under else.
- With score = 70, score >= 60 is true → prints pass ✅

Last tiny piece for this lesson: the "not equal" operator.

Python has:

- == → equal to
- != → not equal to

Example:

Python

```
age = 10

if age != 10:
    print("not ten")
else:
    print("exactly ten")
```

Here, age != 10 means "age is anything except 10".

Make sense that != is the "not equal" comparison operator?

Yes, that makes sense

What do you think? ?

Which operator in Python means “not equal to”?

A.

==

B.

!=

C.

=

D.

>=

Nice ? You nailed it — != is exactly the “not equal to” operator.

- == asks: “are these the same?”
- != asks: “are these different?”
- = is only for assignment, not for comparing.

Example:

```
Python
x = 5

if x != 10:
    print("x is not ten")
```

This will print "x is not ten" because 5 is not equal to 10.

You’ve now used:

- indentation to define blocks
- if/else with a comparison
- =, ==, and != correctly

You’re hitting all the goals for this lesson ?

Do you want to do one more tiny practice with a different comparison (like < or >=), or are you ready to wrap up this lesson and move on?

I'm ready to move on

Awesome work on this lesson, Ron ?

Here’s what you’ve locked in:

- Indentation = blocks: You saw that lines with the same indent belong to the same block (if block vs else block).
- Basic if/else: You wrote a clean if/else with >= to decide pass/fail, and you understood that only one branch runs.
- = vs == vs !=:
- = assigns a value (score = 70)

- == checks equality (if score == 70:)
- != checks not-equal (if score != 70:)

You’re ready for deeper decision-making and, soon, loops ?

When you’re ready, go ahead and hit Next Lesson in the Learn tab, and I’ll be there to walk through the next step with you.

How would you rate this lesson?